263 Project

Hannah Hamling

2026-04-30

Read in data/variable selection

```
library(readxl)
library(dplyr)
fetal.dat <- read_excel("../Data/raw.fetal.data.xlsx")
fetal.dat <- fetal.dat[-1,] #empty first row
#each row is its own id
#variables we'll actually use (including NSP)
#removing date, id, filename, FEB, b, e, AC, DR, SUSP, class, and other classification variables not in
fetal <- fetal.dat |>
  dplyr::select(-c("FileName","Date","SegFile","b", "e", "LBE", "AC", "DR", "A",
                   "B", "C", "D", "E", "AD","DE","LD","FS","SUSP","CLASS"))
#make NSP a factor (only categorical variable in our selected features)
fetal$NSP <- factor(fetal$NSP)

dim(fetal) #2129 rows (people), 20 features, 1 outcome variable (NSP)
```

```
## [1] 2129   21
```

```
#2 rows missing data on every variable: can remove
#last row missing some variables, doesn't have a classification label
fetal <- fetal[-c(2127,2128),]
dim(fetal) #2127 people, 20 features, 1 outcome variable
```

```
## [1] 2127   21
```

```
fetal[!complete.cases(fetal), ] #only this one row missing data
```

```
## # A tibble: 1 x 21
##      LB    FM    UC  ASTV  MSTV  ALTV  MLTV    DL    DS    DP Width   Min   Max
##   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1    NA   564    23    87     7    91  50.7    16     1     4    NA    NA    NA
## # i 8 more variables: Nmax <dbl>, Nzeros <dbl>, Mode <dbl>, Mean <dbl>,
## #   Median <dbl>, Variance <dbl>, Tendency <dbl>, NSP <fct>
```

Look at class frequencies (for later class minority adjustment)

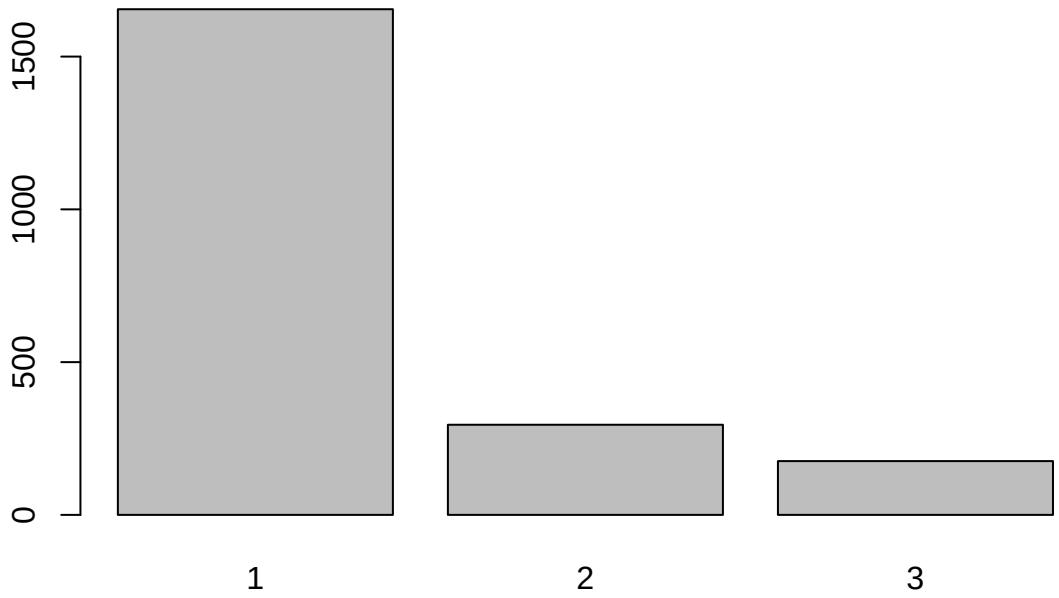
```r
summary(fetal$NSP)
```

```
##    1    2    3 NA's
## 1655  295  176    1
```

```r
#1655 are normal, 295 are suspect, 176 are pathologic, 1 missing
#plot(fetal$NSP)

# drop the 1 observation with NA for NSP -> 2126 observations
fetal <- fetal |> drop_na(NSP)

#1655 are normal, 295 are suspect, 176 are pathologic
plot(fetal$NSP)
```




```r
library(caret)
```

```
## Loading required package: lattice

##
## Attaching package: 'caret'

## The following object is masked from 'package:purrr':
##
##     lift
```

```r
set.seed(263)

train_indices <- createDataPartition(fetal$NSP, p = 0.8, list = FALSE)

fetal_train <- fetal[train_indices, ]
fetal_test  <- fetal[-train_indices, ]

# make sure there are some of each NSP in both train and test data
table(fetal_train$NSP)
```

```
##
##    1    2    3
## 1324  236  141
```

```r
table(fetal_test$NSP)
```

```
##
##   1   2   3
## 331  59  35
```

Custom Asymmetric Loss Analysis

We have a classification problem with 3 classes of fetal state (1 = Normal, 2 = Suspect, 3 = Pathological). Classes 2 and 3 are underrepresented compared to class 1. Clinically, certain misclassifications are more harmful than others.

- Mistaking 3 as 1: Worst possible mistake
- Mistaking 2 as 1: Also a pretty bad mistake
- Mistaking 3 as 2: Bad, but not as bad as the previous two cases
- Mistaking 2 as 3: not that bad
- Mistaking 1 as 2: not that bad
- Mistaking 1 as 3: not that bad

Thus, we can define a custom loss that punishes certain mixups accordingly.

```r
# row is true class, column is predicted class

# all misclassifications are punished equally
loss_default <- matrix(c(0, 1, 1,
                         1, 0, 1,
                         1, 1, 0), nrow = 3, byrow = TRUE)

# we can change these values, like idk is a false negative 4x as bad as a false positive
loss_mild <- matrix(c(0, 1, 1,
                      3, 0, 1,
                      4, 2, 0), nrow = 3, byrow = TRUE)

loss_aggressive <- matrix(c(0, 1, 1,
                            12, 0, 1,
                            12, 3, 0), nrow = 3, byrow = TRUE)
```

```r
calculate_loss <- function(true_labels, predicted_labels, loss_matrix) {
  true_labels <- as.numeric(true_labels)
  predicted_labels <- as.numeric(predicted_labels)
  mean(mapply(FUN = function(t, p) loss_matrix[t, p], true_labels, predicted_labels))
}
```

Default Single Tree Using All Variables

Let's see how a single basic tree with all variables and default arguments does with our custom loss. We see that 2 gets misclassified as 1 a lot (bad mistake), which increases the loss.

```r
library(rpart)
library(rpart.plot)

set.seed(263)

basic_tree <- rpart(NSP ~ . , data = fetal_train, method = "class")

#rpart.plot(basic_tree)

# training confusion matrix
basic_tree_train_prediction <- predict(basic_tree, fetal_train, type = "class")
confusionMatrix(basic_tree_train_prediction, fetal_train$NSP) # 93.36 train acc
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    1    2    3
##          1 1304   78    8
##          2   11  153    2
##          3    9    5  131
##
## Overall Statistics
##
##                Accuracy : 0.9336
##                  95% CI : (0.9207, 0.9449)
##     No Information Rate : 0.7784
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.8065
##
##  Mcnemar's Test P-Value : 3.332e-11
##
## Statistics by Class:
##
##                      Class: 1 Class: 2 Class: 3
## Sensitivity            0.9849  0.64831  0.92908
## Specificity            0.7719  0.99113  0.99103
## Pos Pred Value         0.9381  0.92169  0.90345
## Neg Pred Value         0.9357  0.94593  0.99357
## Prevalence             0.7784  0.13874  0.08289
## Detection Rate         0.7666  0.08995  0.07701
```

```
## Detection Prevalence   0.8172  0.09759  0.08524
## Balanced Accuracy      0.8784  0.81972  0.96005
```

```
# test confusion matrix
basic_tree_test_prediction <- predict(basic_tree, fetal_test, type = "class")
confusionMatrix(basic_tree_test_prediction, fetal_test$NSP) # 92.47 test acc
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   1   2   3
##          1 326  24   2
##          2   5  35   1
##          3   0   0  32
##
## Overall Statistics
##
##                Accuracy : 0.9247
##                  95% CI : (0.8954, 0.9479)
##     No Information Rate : 0.7788
##     P-Value [Acc > NIR] : 4.783e-16
##
##                   Kappa : 0.7755
##
##  Mcnemar's Test P-Value : 0.001471
##
## Statistics by Class:
##
##                      Class: 1 Class: 2 Class: 3
## Sensitivity            0.9849  0.59322  0.91429
## Specificity            0.7234  0.98361  1.00000
## Pos Pred Value         0.9261  0.85366  1.00000
## Neg Pred Value         0.9315  0.93750  0.99237
## Prevalence             0.7788  0.13882  0.08235
## Detection Rate         0.7671  0.08235  0.07529
## Detection Prevalence   0.8282  0.09647  0.07529
## Balanced Accuracy      0.8541  0.78841  0.95714
```

```
# model predicts a lot of 2's as 1's, so we should punish this more

# training losses
calculate_loss(basic_tree_train_prediction, fetal_train$NSP, loss_default)
```

```
## [1] 0.06643151
```

```
calculate_loss(basic_tree_train_prediction, fetal_train$NSP, loss_mild)
```

```
## [1] 0.09817754
```

```
calculate_loss(basic_tree_train_prediction, fetal_train$NSP, loss_aggressive)
```

```
## [1] 0.2016461
```

```r
# test losses
calculate_loss(basic_tree_test_prediction, fetal_test$NSP, loss_default)
```

```
## [1] 0.07529412
```

```r
calculate_loss(basic_tree_test_prediction, fetal_test$NSP, loss_mild)
```

```
## [1] 0.09882353
```

```r
calculate_loss(basic_tree_test_prediction, fetal_test$NSP, loss_aggressive)
```

```
## [1] 0.2047059
```

Default RF

The default RF model shows a large class error rate for class 2 (and 3 somewhat), so the loss increases when we use our custom loss.

```r
library(randomForest)
set.seed(263)
rf_default <- randomForest(NSP ~ ., data = fetal_train)
rf_default # once again, 2 is being misclassified as 1, OOB error = 5.7%
```

```
##
## Call:
##  randomForest(formula = NSP ~ ., data = fetal_train)
##                Type of random forest: classification
##                      Number of trees: 500
## No. of variables tried at each split: 4
##
##         OOB estimate of  error rate: 5.7%
## Confusion matrix:
##      1   2   3 class.error
## 1 1310  11   3  0.01057402
## 2   60 172   4  0.27118644
## 3    8  11 122  0.13475177
```

```r
# train losses
calculate_loss(predict(rf_default, fetal_train), fetal_train$NSP, loss_default)
```

```
## [1] 0.001175779
```

```r
calculate_loss(predict(rf_default, fetal_train), fetal_train$NSP, loss_mild)
```

```
## [1] 0.002351558
```

```r
calculate_loss(predict(rf_default, fetal_train), fetal_train$NSP, loss_aggressive)
```

```
## [1] 0.007642563
```

```r
# test losses
calculate_loss(predict(rf_default, fetal_test), fetal_test$NSP, loss_default)
```

```
## [1] 0.05882353
```

```r
calculate_loss(predict(rf_default, fetal_test), fetal_test$NSP, loss_mild)
```

```
## [1] 0.08470588
```

```r
calculate_loss(predict(rf_default, fetal_test), fetal_test$NSP, loss_aggressive)
```

```
## [1] 0.1929412
```

Default Boosting

Boosting model also misclassifies 2 as 1 a lot. 3 as 1 too sometimes.

```r
library(gbm)

get_gbm_labels <- function(gbm_model, dataset, n.trees = 100) { # default n.trees for gbm is 100
  as.factor(max.col((predict.gbm(gbm_model, dataset, type = "response", n.trees = n.trees)[,,1])))
}

set.seed(263)
gbm_default <- gbm(NSP ~ ., data = fetal_train, distribution = "multinomial")

# training confusion matrix
gbm_default_train_prediction <- get_gbm_labels(gbm_default, fetal_train)
confusionMatrix(gbm_default_train_prediction, fetal_train$NSP) # 94.42 train acc
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    1    2    3
##          1 1299   53   12
##          2   16  182    4
##          3    9    1  125
##
## Overall Statistics
##
##                Accuracy : 0.9442
##                  95% CI : (0.9322, 0.9546)
##     No Information Rate : 0.7784
##     P-Value [Acc > NIR] : < 2.2e-16
##
```

```
##                   Kappa : 0.8417
##
##  Mcnemar's Test P-Value : 6.311e-05
##
## Statistics by Class:
##
##                      Class: 1 Class: 2 Class: 3
## Sensitivity            0.9811   0.7712  0.88652
## Specificity            0.8276   0.9863  0.99359
## Pos Pred Value         0.9523   0.9010  0.92593
## Neg Pred Value         0.9258   0.9640  0.98978
## Prevalence             0.7784   0.1387  0.08289
## Detection Rate         0.7637   0.1070  0.07349
## Detection Prevalence   0.8019   0.1188  0.07937
## Balanced Accuracy      0.9044   0.8788  0.94006
```

```
# test confusion matrix
gbm_default_test_prediction <- get_gbm_labels(gbm_default, fetal_test)
confusionMatrix(gbm_default_test_prediction, fetal_test$NSP) # 92.47 test acc
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   1   2   3
##          1 322  18   2
##          2   9  41   3
##          3   0   0  30
##
## Overall Statistics
##
##                Accuracy : 0.9247
##                  95% CI : (0.8954, 0.9479)
##     No Information Rate : 0.7788
##     P-Value [Acc > NIR] : 4.783e-16
##
##                   Kappa : 0.785
##
##  Mcnemar's Test P-Value : 0.04601
##
## Statistics by Class:
##
##                      Class: 1 Class: 2 Class: 3
## Sensitivity            0.9728  0.69492  0.85714
## Specificity            0.7872  0.96721  1.00000
## Pos Pred Value         0.9415  0.77358  1.00000
## Neg Pred Value         0.8916  0.95161  0.98734
## Prevalence             0.7788  0.13882  0.08235
## Detection Rate         0.7576  0.09647  0.07059
## Detection Prevalence   0.8047  0.12471  0.07059
## Balanced Accuracy      0.8800  0.83106  0.92857
```

```
# train losses
calculate_loss(gbm_default_train_prediction, fetal_train$NSP, loss_default)
```

```
## [1] 0.0558495
```

```
calculate_loss(gbm_default_train_prediction, fetal_train$NSP, loss_mild)
```

```
## [1] 0.09112287
```

```
calculate_loss(gbm_default_train_prediction, fetal_train$NSP, loss_aggressive)
```

```
## [1] 0.2186949
```

```
# test losses
calculate_loss(gbm_default_test_prediction, fetal_test$NSP, loss_default)
```

```
## [1] 0.07529412
```

```
calculate_loss(gbm_default_test_prediction, fetal_test$NSP, loss_mild)
```

```
## [1] 0.1176471
```

```
calculate_loss(gbm_default_test_prediction, fetal_test$NSP, loss_aggressive)
```

```
## [1] 0.3082353
```

RF CV

now doing cv to select the best RF hyperparameters but according to our custom loss (hopefully helps to fix the bad sorts of mixups)

Hyperparameters include: mtry and ntree

```
set.seed(263)
# 5 folds
K <- 5
fold <- sample(1:K, nrow(fetal_train), replace = TRUE)

mtry_values <- c(3, 5, 7, 9) # we have 20 variables, the default is p/3
ntree_values <- c(250, 500, 750, 1000, 1500) # default is 500

rf_cv_results <- data.frame(mtry = c(), ntree = c(), loss_default = c(), loss_mild = c(), loss_aggressi

for(m in mtry_values) {
  for(n in ntree_values) {
    fold_loss_default <- c()
    fold_loss_mild <- c()
    fold_loss_aggressive <- c()

    for(k in 1:K) {
      train <- fetal_train[fold != k, ]
      val <- fetal_train[fold == k, ]
```

```
      rf_model <- randomForest(NSP ~ .,
                               data = train,
                               mtry = m,
                               ntree = n)

      fold_loss_default[k] <- calculate_loss(val$NSP, predict(rf_model, val), loss_default)
      fold_loss_mild[k] <- calculate_loss(val$NSP, predict(rf_model, val), loss_mild)
      fold_loss_aggressive[k] <- calculate_loss(val$NSP, predict(rf_model, val), loss_aggressive)
    }

    rf_cv_results <- rf_cv_results |> rbind(data.frame(
      mtry = m,
      ntree = n,
      loss_default = mean(fold_loss_default),
      loss_mild = mean(fold_loss_mild),
      loss_aggressive = mean(fold_loss_aggressive))
    )

  }
}

rf_cv_results
```

```
##    mtry ntree loss_default loss_mild loss_aggressive
## 1     3   250   0.06471110 0.2033978       0.3404209
## 2     3   500   0.06953737 0.2152796       0.3593001
## 3     3   750   0.06944408 0.2184347       0.3662417
## 4     3  1000   0.06892663 0.2158124       0.3609621
## 5     3  1500   0.06828350 0.2151149       0.3602102
## 6     5   250   0.06137966 0.1884803       0.3174143
## 7     5   500   0.06243638 0.1952216       0.3252268
## 8     5   750   0.06127334 0.1929911       0.3254686
## 9     5  1000   0.06134463 0.1937743       0.3281063
## 10    5  1500   0.06073776 0.1878915       0.3157845
## 11    7   250   0.06137947 0.1914493       0.3228314
## 12    7   500   0.06082086 0.1874633       0.3154181
## 13    7   750   0.06077279 0.1873028       0.3151451
## 14    7  1000   0.06134199 0.1912993       0.3225690
## 15    7  1500   0.06192950 0.1884595       0.3163018
## 16    9   250   0.06253638 0.1924937       0.3237634
## 17    9   500   0.06196699 0.1925143       0.3249639
## 18    9   750   0.06141450 0.1897144       0.3193266
## 19    9  1000   0.06255696 0.1932167       0.3257788
## 20    9  1500   0.06141450 0.1897144       0.3193266
```

GBM CV

now doing cv to select the best GBM hyperparameters but according to our custom loss (hopefully helps to fix the bad sorts of mixups)

Hyperparameters include: n.trees, interaction.depth, shrinkage

```r
set.seed(263)
gbm_cv_results <- data.frame(n.trees = c(), interaction.depth = c(), shrinkage = c(), loss_default = c()

n.trees_values <- c(50, 100, 250, 500) # default is 100
interaction.depth_values <- c(1, 2, 3, 4, 5) # default is a depth of 1
shrinkage_values <- c(0.01, 0.1, 0.2) # default is 0.1

for(n in n.trees_values) {
  for(d in interaction.depth_values) {
    for(s in shrinkage_values) {
      fold_loss_default <- c()
      fold_loss_mild <- c()
      fold_loss_aggressive <- c()

      for(k in 1:K) {
        train <- fetal_train[fold != k, ]
        val <- fetal_train[fold == k, ]

        gbm_model <- gbm(NSP ~ ., data = train, distribution = "multinomial",
                         n.trees = n, interaction.depth = d, shrinkage = s)


        val_predicted_label <- get_gbm_labels(gbm_model, val, n.trees = n)

        fold_loss_default[k] <- calculate_loss(val$NSP, val_predicted_label, loss_default)
        fold_loss_mild[k] <- calculate_loss(val$NSP, val_predicted_label, loss_mild)
        fold_loss_aggressive[k] <- calculate_loss(val$NSP, val_predicted_label, loss_aggressive)
      }
    gbm_cv_results <- gbm_cv_results |> rbind(data.frame(
      n.trees = n,
      interaction.depth = d,
      shrinkage = s,
      loss_default = mean(fold_loss_default),
      loss_mild = mean(fold_loss_mild),
      loss_aggressive = mean(fold_loss_aggressive))
    )
    }
  }
}

gbm_cv_results
```

```
##    n.trees interaction.depth shrinkage loss_default loss_mild loss_aggressive
## 1       50                 1      0.01   0.12988675 0.3474250       1.1032638
## 2       50                 1      0.10   0.07654675 0.1953819       0.6660512
## 3       50                 1      0.20   0.07394325 0.1670662       0.5351894
## 4       50                 2      0.01   0.09463507 0.2531038       0.8538598
## 5       50                 2      0.10   0.06477568 0.1608697       0.5391176
## 6       50                 2      0.20   0.06645126 0.1511887       0.4827232
## 7       50                 3      0.01   0.08343953 0.2200475       0.7681400
## 8       50                 3      0.10   0.06372632 0.1587571       0.5373060
## 9       50                 3      0.20   0.06424964 0.1462161       0.4695442
## 10      50                 4      0.01   0.07751905 0.2048785       0.7165066
```

```
##  11      50                4      0.10   0.06404190 0.1501567       0.4941583
##  12      50                4      0.20   0.06141469 0.1435168       0.4687667
##  13      50                5      0.01   0.07517445 0.2001070       0.7103103
##  14      50                5      0.10   0.06556100 0.1512763       0.4914743
##  15      50                5      0.20   0.06413416 0.1505222       0.4940320
##  16     100                1      0.01   0.12356696 0.3449940       1.1229549
##  17     100                1      0.10   0.06756888 0.1652731       0.5473295
##  18     100                1      0.20   0.07055066 0.1587756       0.5101867
##  19     100                2      0.01   0.08739840 0.2313233       0.7815549
##  20     100                2      0.10   0.06681564 0.1540337       0.4994840
##  21     100                2      0.20   0.06019972 0.1393300       0.4466640
##  22     100                3      0.01   0.07684598 0.2091879       0.7349313
##  23     100                3      0.10   0.06061105 0.1435906       0.4717971
##  24     100                3      0.20   0.06362464 0.1454135       0.4628808
##  25     100                4      0.01   0.07203660 0.1936344       0.6863636
##  26     100                4      0.10   0.06482761 0.1527993       0.4970738
##  27     100                4      0.20   0.06027224 0.1359035       0.4339680
##  28     100                5      0.01   0.06911934 0.1867057       0.6645351
##  29     100                5      0.10   0.06363322 0.1479588       0.4819030
##  30     100                5      0.20   0.06356109 0.1431414       0.4622071
##  31     250                1      0.01   0.09845913 0.2685254       0.9166025
##  32     250                1      0.10   0.06524280 0.1472437       0.4707617
##  33     250                1      0.20   0.07133191 0.1588751       0.5095105
##  34     250                2      0.01   0.07124476 0.1858454       0.6408970
##  35     250                2      0.10   0.06019850 0.1404491       0.4594137
##  36     250                2      0.20   0.06304363 0.1489305       0.4891469
##  37     250                3      0.01   0.06720052 0.1743401       0.5988558
##  38     250                3      0.10   0.06004308 0.1367846       0.4398453
##  39     250                3      0.20   0.06377457 0.1428253       0.4561843
##  40     250                4      0.01   0.06587169 0.1692594       0.5846272
##  41     250                4      0.10   0.05718000 0.1332470       0.4364244
##  42     250                4      0.20   0.05784454 0.1335573       0.4320668
##  43     250                5      0.01   0.06525668 0.1668729       0.5775920
##  44     250                5      0.10   0.05615019 0.1340040       0.4420702
##  45     250                5      0.20   0.06135379 0.1428823       0.4674316
##  46     500                1      0.01   0.07429687 0.1932436       0.6645295
##  47     500                1      0.10   0.06946872 0.1550146       0.4946560
##  48     500                1      0.20   0.07085234 0.1601993       0.5113005
##  49     500                2      0.01   0.06530513 0.1618899       0.5454980
##  50     500                2      0.10   0.06540862 0.1467184       0.4640525
##  51     500                2      0.20   0.06554344 0.1510041       0.4905469
##  52     500                3      0.01   0.06592994 0.1596567       0.5295443
##  53     500                3      0.10   0.05959814 0.1402645       0.4577424
##  54     500                3      0.20   0.06427286 0.1438892       0.4569265
##  55     500                4      0.01   0.06412764 0.1542393       0.5142205
##  56     500                4      0.10   0.05960465 0.1385533       0.4518305
##  57     500                4      0.20           NA        NA              NA
##  58     500                5      0.01   0.06285829 0.1493588       0.4932962
##  59     500                5      0.10   0.06129430 0.1433142       0.4668827
##  60     500                5      0.20           NA        NA              NA
```

Choosing Best Model based on Custom Loss

We choose to sacrifice overall accuracy in order to improve in positive identification of classes 2 and 3.

Default Loss (equal punishment for all misclassifications)

Best RF model with default loss is: ntree = 1500, mtry = 5

```
set.seed(263)
best_rf_default_loss <- rf_cv_results[which.min(rf_cv_results$loss_default), ]
best_rf_default_loss
```

```
##    mtry ntree loss_default loss_mild loss_aggressive
## 10    5  1500   0.06073776 0.1878915       0.3157845
```

```
best_rf_default_loss_ntree <- best_rf_default_loss$ntree
best_rf_default_loss_mtry <- best_rf_default_loss$mtry
best_rf_default_loss_model <- randomForest(NSP ~., data = fetal_train,
                                           ntree = best_rf_default_loss_ntree,
                                           mtry = best_rf_default_loss_mtry)
best_rf_default_loss_model # OOB error = 5.29%, lots of 2's classified as 1's
```

```
##
## Call:
##  randomForest(formula = NSP ~ ., data = fetal_train, ntree = best_rf_default_loss_ntree,      mtry =
##                Type of random forest: classification
##                      Number of trees: 1500
## No. of variables tried at each split: 5
##
##         OOB estimate of  error rate: 5.29%
## Confusion matrix:
##      1   2   3 class.error
## 1 1310  10   4  0.01057402
## 2   59 173   4  0.26694915
## 3    5   8 128  0.09219858
```

```
calculate_loss(fetal_train$NSP, predict(best_rf_default_loss_model, fetal_train), loss_default)
```

```
## [1] 0.001175779
```

```
calculate_loss(fetal_test$NSP, predict(best_rf_default_loss_model, fetal_test), loss_default)
```

```
## [1] 0.06117647
```

Best GBM with default loss is n.trees = 250, interaction.depth = 5, shrinkage = 0.1

```
set.seed(263)
best_gbm_default_loss <- gbm_cv_results[which.min(gbm_cv_results$loss_default), ]
best_gbm_default_loss
```

```
##    n.trees interaction.depth shrinkage loss_default loss_mild loss_aggressive
## 44     250                 5       0.1   0.05615019  0.134004       0.4420702
```

```
best_gbm_default_loss_n.trees <- best_gbm_default_loss$n.trees
best_gbm_default_loss_interaction.depth <- best_gbm_default_loss$interaction.depth
best_gbm_default_loss_shrinkage <- best_gbm_default_loss$shrinkage
best_gbm_default_loss_model <- gbm(NSP ~., data = fetal_train,
                                   n.trees = best_gbm_default_loss_n.trees,
                                   interaction.depth = best_gbm_default_loss_interaction.depth,
                                   shrinkage = best_gbm_default_loss_shrinkage)
```

```
## Distribution not specified, assuming multinomial ...
```

```
## Warning: Setting `distribution = "multinomial"` is ill-advised as it is
## currently broken. It exists only for backwards compatibility. Use at your own
## risk.
```

```
confusionMatrix(get_gbm_labels(best_gbm_default_loss_model, fetal_train, n.trees = best_gbm_default_loss
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    1    2    3
##          1 1323    0    0
##          2    1  236    0
##          3    0    0  141
##
## Overall Statistics
##
##                Accuracy : 0.9994
##                  95% CI : (0.9967, 1)
##     No Information Rate : 0.7784
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.9984
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: 1 Class: 2 Class: 3
## Sensitivity            0.9992   1.0000  1.00000
## Specificity            1.0000   0.9993  1.00000
## Pos Pred Value         1.0000   0.9958  1.00000
## Neg Pred Value         0.9974   1.0000  1.00000
## Prevalence             0.7784   0.1387  0.08289
## Detection Rate         0.7778   0.1387  0.08289
## Detection Prevalence   0.7778   0.1393  0.08289
## Balanced Accuracy      0.9996   0.9997  1.00000
```

```r
confusionMatrix(get_gbm_labels(best_gbm_default_loss_model, fetal_test, n.trees = best_gbm_default_loss
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   1   2   3
##          1 321  13   2
##          2   9  46   2
##          3   1   0  31
##
## Overall Statistics
##
##                Accuracy : 0.9365
##                  95% CI : (0.9089, 0.9577)
##     No Information Rate : 0.7788
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.8233
##
##  Mcnemar's Test P-Value : 0.3824
##
## Statistics by Class:
##
##                      Class: 1 Class: 2 Class: 3
## Sensitivity            0.9698   0.7797  0.88571
## Specificity            0.8404   0.9699  0.99744
## Pos Pred Value         0.9554   0.8070  0.96875
## Neg Pred Value         0.8876   0.9647  0.98982
## Prevalence             0.7788   0.1388  0.08235
## Detection Rate         0.7553   0.1082  0.07294
## Detection Prevalence   0.7906   0.1341  0.07529
## Balanced Accuracy      0.9051   0.8748  0.94158
```

```r
calculate_loss(fetal_train$NSP, get_gbm_labels(best_gbm_default_loss_model, fetal_train, n.trees = best
```

```
## [1] 0.0005878895
```

```r
calculate_loss(fetal_test$NSP, get_gbm_labels(best_gbm_default_loss_model, fetal_test, n.trees = best_g
```

```
## [1] 0.06352941
```

Mild Loss

Best RF model with mild loss is: ntree = 750, mtry = 7

```r
set.seed(263)
best_rf_mild_loss <- rf_cv_results[which.min(rf_cv_results$loss_mild), ]
best_rf_mild_loss
```

```
##    mtry ntree loss_default loss_mild loss_aggressive
## 13    7   750   0.06077279 0.1873028       0.3151451
```

```r
best_rf_mild_loss_ntree <- best_rf_mild_loss$ntree
best_rf_mild_loss_mtry <- best_rf_mild_loss$mtry
best_rf_mild_loss_model <- randomForest(NSP ~., data = fetal_train,
                                        ntree = best_rf_mild_loss_ntree,
                                        mtry = best_rf_mild_loss_mtry)
best_rf_mild_loss_model # OOB error = 5.47%, still pretty bad at 2's but better at 3's than default los
```

```
##
## Call:
##  randomForest(formula = NSP ~ ., data = fetal_train, ntree = best_rf_mild_loss_ntree,      mtry = bes
##                Type of random forest: classification
##                      Number of trees: 750
## No. of variables tried at each split: 7
##
##         OOB estimate of  error rate: 5.47%
## Confusion matrix:
##      1   2   3 class.error
## 1 1307  12   5  0.01283988
## 2   61 171   4  0.27542373
## 3    5   6 130  0.07801418
```

```r
calculate_loss(fetal_train$NSP, predict(best_rf_mild_loss_model, fetal_train), loss_mild)
```

```
## [1] 0.0005878895
```

```r
calculate_loss(fetal_test$NSP, predict(best_rf_mild_loss_model, fetal_test), loss_mild)
```

```
## [1] 0.1576471
```

Best GBM with mild loss is n.trees = 250, interaction.depth = 4, shrinkage = 0.1

```r
set.seed(263)
best_gbm_mild_loss <- gbm_cv_results[which.min(gbm_cv_results$loss_mild), ]
best_gbm_mild_loss
```

```
##    n.trees interaction.depth shrinkage loss_default loss_mild loss_aggressive
## 71     500                 3       0.1   0.05665074 0.1316466       0.4300037
```

```r
best_gbm_mild_loss_n.trees <- best_gbm_mild_loss$n.trees
best_gbm_mild_loss_interaction.depth <- best_gbm_mild_loss$interaction.depth
best_gbm_mild_loss_shrinkage <- best_gbm_mild_loss$shrinkage
best_gbm_mild_loss_model <- gbm(NSP ~., data = fetal_train,
                                n.trees = best_gbm_mild_loss_n.trees,
                                interaction.depth = best_gbm_mild_loss_interaction.depth,
                                shrinkage = best_gbm_mild_loss_shrinkage)
```

```
## Distribution not specified, assuming multinomial ...

## Warning: Setting `distribution = "multinomial"` is ill-advised as it is
## currently broken. It exists only for backwards compatibility. Use at your own
## risk.
```

```
confusionMatrix(get_gbm_labels(best_gbm_mild_loss_model, fetal_train, n.trees = best_gbm_mild_loss_n.tr
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    1    2    3
##          1 1323    0    0
##          2    1  236    0
##          3    0    0  141
##
## Overall Statistics
##
##                Accuracy : 0.9994
##                  95% CI : (0.9967, 1)
##     No Information Rate : 0.7784
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.9984
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: 1 Class: 2 Class: 3
## Sensitivity            0.9992   1.0000  1.00000
## Specificity            1.0000   0.9993  1.00000
## Pos Pred Value         1.0000   0.9958  1.00000
## Neg Pred Value         0.9974   1.0000  1.00000
## Prevalence             0.7784   0.1387  0.08289
## Detection Rate         0.7778   0.1387  0.08289
## Detection Prevalence   0.7778   0.1393  0.08289
## Balanced Accuracy      0.9996   0.9997  1.00000
```

```
confusionMatrix(get_gbm_labels(best_gbm_mild_loss_model, fetal_test, n.trees = best_gbm_mild_loss_n.tre
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   1   2   3
##          1 322   9   2
##          2   8  48   2
##          3   1   2  31
##
## Overall Statistics
##
##                Accuracy : 0.9435
##                  95% CI : (0.9171, 0.9635)
##     No Information Rate : 0.7788
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.845
##
```

```
##  Mcnemar's Test P-Value : 0.9419
##
## Statistics by Class:
##
##                      Class: 1 Class: 2 Class: 3
## Sensitivity            0.9728   0.8136  0.88571
## Specificity            0.8830   0.9727  0.99231
## Pos Pred Value         0.9670   0.8276  0.91176
## Neg Pred Value         0.9022   0.9700  0.98977
## Prevalence             0.7788   0.1388  0.08235
## Detection Rate         0.7576   0.1129  0.07294
## Detection Prevalence   0.7835   0.1365  0.08000
## Balanced Accuracy      0.9279   0.8931  0.93901
```

```
calculate_loss(fetal_train$NSP, get_gbm_labels(best_gbm_mild_loss_model, fetal_train, n.trees = best_gbm
```

```
## [1] 0.0005878895
```

```
calculate_loss(fetal_test$NSP, get_gbm_labels(best_gbm_mild_loss_model, fetal_test, n.trees = best_gbm_
```

```
## [1] 0.1176471
```

Aggressive Loss

Best RF model with aggressive loss is: ntree = 750, mtry = 7, same as mild loss best RF hyperparameters

```
set.seed(263)
best_rf_aggressive_loss <- rf_cv_results[which.min(rf_cv_results$loss_aggressive), ]
best_rf_aggressive_loss
```

```
##    mtry ntree loss_default loss_mild loss_aggressive
## 13    7   750   0.06077279 0.1873028       0.3151451
```

```
best_rf_aggressive_loss_ntree <- best_rf_aggressive_loss$ntree
best_rf_aggressive_loss_mtry <- best_rf_aggressive_loss$mtry
best_rf_aggressive_loss_model <- randomForest(NSP ~., data = fetal_train,
                                              ntree = best_rf_aggressive_loss_ntree,
                                              mtry = best_rf_aggressive_loss_mtry)
best_rf_aggressive_loss_model # OOB error = 5.47%, still pretty bad at 2's but better at 3's than defau
```

```
##
## Call:
##  randomForest(formula = NSP ~ ., data = fetal_train, ntree = best_rf_aggressive_loss_ntree,      mtry
##                Type of random forest: classification
##                      Number of trees: 750
## No. of variables tried at each split: 7
##
##         OOB estimate of  error rate: 5.47%
## Confusion matrix:
##      1   2   3 class.error
## 1 1307  12   5  0.01283988
## 2   61 171   4  0.27542373
## 3    5   6 130  0.07801418
```

```r
calculate_loss(fetal_train$NSP, predict(best_rf_aggressive_loss_model, fetal_train), loss_aggressive)
```

```
## [1] 0.0005878895
```

```r
calculate_loss(fetal_test$NSP, predict(best_rf_aggressive_loss_model, fetal_test), loss_aggressive)
```

```
## [1] 0.5576471
```

Best GBM with aggressive loss is n.trees = 250, interaction.depth = 4, shrinkage = 0.2. This model has the most balanced accuracy

```r
set.seed(263)
best_gbm_aggressive_loss <- gbm_cv_results[which.min(gbm_cv_results$loss_aggressive), ]
best_gbm_aggressive_loss
```

```
##    n.trees interaction.depth shrinkage loss_default loss_mild loss_aggressive
## 42     250                 4       0.2   0.05784454 0.1335573       0.4320668
```

```r
best_gbm_aggressive_loss_n.trees <- best_gbm_aggressive_loss$n.trees
best_gbm_aggressive_loss_interaction.depth <- best_gbm_aggressive_loss$interaction.depth
best_gbm_aggressive_loss_shrinkage <- best_gbm_aggressive_loss$shrinkage
best_gbm_aggressive_loss_model <- gbm(NSP ~., data = fetal_train,
                                      n.trees = best_gbm_aggressive_loss_n.trees,
                                      interaction.depth = best_gbm_aggressive_loss_interaction.depth,
                                      shrinkage = best_gbm_aggressive_loss_shrinkage)
```

```
## Distribution not specified, assuming multinomial ...
```

```
## Warning: Setting `distribution = "multinomial"` is ill-advised as it is
## currently broken. It exists only for backwards compatibility. Use at your own
## risk.
```

```r
confusionMatrix(get_gbm_labels(best_gbm_aggressive_loss_model, fetal_train, n.trees = best_gbm_aggressi
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    1    2    3
##          1 1323    0    0
##          2    1  236    0
##          3    0    0  141
##
## Overall Statistics
##
##                Accuracy : 0.9994
##                  95% CI : (0.9967, 1)
##     No Information Rate : 0.7784
##     P-Value [Acc > NIR] : < 2.2e-16
##
```

```
##                  Kappa : 0.9984
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: 1 Class: 2 Class: 3
## Sensitivity            0.9992   1.0000  1.00000
## Specificity            1.0000   0.9993  1.00000
## Pos Pred Value         1.0000   0.9958  1.00000
## Neg Pred Value         0.9974   1.0000  1.00000
## Prevalence             0.7784   0.1387  0.08289
## Detection Rate         0.7778   0.1387  0.08289
## Detection Prevalence   0.7778   0.1393  0.08289
## Balanced Accuracy      0.9996   0.9997  1.00000
```

```
confusionMatrix(get_gbm_labels(best_gbm_aggressive_loss_model, fetal_test, n.trees = best_gbm_aggressive
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   1   2   3
##          1 322   9   2
##          2   8  49   2
##          3   1   1  31
##
## Overall Statistics
##
##                Accuracy : 0.9459
##                  95% CI : (0.9199, 0.9654)
##     No Information Rate : 0.7788
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.8514
##
##  Mcnemar's Test P-Value : 0.8672
##
## Statistics by Class:
##
##                      Class: 1 Class: 2 Class: 3
## Sensitivity            0.9728   0.8305  0.88571
## Specificity            0.8830   0.9727  0.99487
## Pos Pred Value         0.9670   0.8305  0.93939
## Neg Pred Value         0.9022   0.9727  0.98980
## Prevalence             0.7788   0.1388  0.08235
## Detection Rate         0.7576   0.1153  0.07294
## Detection Prevalence   0.7835   0.1388  0.07765
## Balanced Accuracy      0.9279   0.9016  0.94029
```

```
calculate_loss(fetal_train$NSP, get_gbm_labels(best_gbm_aggressive_loss_model, fetal_train, n.trees = b
```

```
## [1] 0.0005878895
```

```
calculate_loss(fetal_test$NSP, get_gbm_labels(best_gbm_aggressive_loss_model, fetal_test, n.trees = bes
```

```
## [1] 0.3482353
```